

გაკვეთილი 18 mod 3 (3.3)

■ გაკვეთილი 6 — `sort()`, `find()`, `filter()`, `map()`

■ რა არის ფუნქცია?

წარმოიდგინე, რომ ყოველდღე ერთსა და იმავე საქმეს აკეთებ.

მაგალითად:

- მიესალმები ადამიანს
- ანგარიშობ ორ რიცხვს
- ბეჭდავ ერთსა და იმავე ტექსტს

ამისთვის კოდის ყოველ ჯერზე თავიდან დაწერა არ გვინდა.

ამიტომ ვქმნით ფუნქციას.

✓ ფუნქციის განმარტება

ფუნქცია (Function) არის კოდის ბლოკი, რომელსაც სახელი აქვს და რომლის გამოძახებაც რამდენჯერაც გვინდა შეგვიძლია.

პირველი ფუნქცია

```
function sayHello(){  
  console.log("გამარჯობა");}
```

აქ:

- `function` → ვეუბნებით JavaScript-ს, რომ ფუნქციას ვქმნით
- `sayHello` → ფუნქციის სახელია
- `{ }` → ფუნქციის სხეულია

რატომ არაფერი გამოიტანა?

თუ მხოლოდ ფუნქციას შევქმნით:

```
function sayHello(){  
  console.log("გამარჯობა");  
}
```

არაფერი მოხდება.

ფუნქცია ჯერ მხოლოდ შეიქმნა.

უნდა გამოვიძახოთ.

ფუნქციის გამოძახება

```
function sayHello(){  
  console.log("გამარჯობა");  
}  
  
sayHello();
```

შედეგი:

გამარჯობა

რამდენჯერაც გვინდა იმდენჯერ გამოვიძახებთ

```
function sayHello(){  
  console.log("გამარჯობა");  
}  
  
sayHello();  
sayHello();  
sayHello();
```

შედეგი:

გამარჯობა
გამარჯობა
გამარჯობა

რატომ არის ფუნქცია კარგი?

ამის ნაცვლად:

```
console.log("გამარჯობა");  
console.log("გამარჯობა");  
console.log("გამარჯობა");
```

ვწერთ:

```
function sayHello(){  
  console.log("გამარჯობა");  
}sayHello();  
sayHello();  
sayHello();
```

კოდი უფრო სუფთაა.

ფუნქციის პარამეტრები

ფუნქციას შეგვიძლია ინფორმაცია გადავცეთ.

მაგალითი

```
function greet(name){  
  console.log("გამარჯობა " + name);  
}  
greet("გიორგი");
```

შედეგი:

```
გამარჯობა გიორგი
```

რა არის `name` ?

```
function greet(name)
```

name არის პარამეტრი.

ანუ ადგილი, სადაც მნიშვნელობა ჩაჯდება.

კიდევ მაგალითი

```
function greet(name){  
  console.log("გამარჯობა " + name);  
}  
greet("ანა");  
greet("ლუკა");  
greet("ნინო");
```

შედეგი:

```
გამარჯობა ანა  
გამარჯობა ლუკა  
გამარჯობა ნინო
```



ორი პარამეტრი

```
function add(a, b){  
  console.log(a + b);  
}  
add(5, 10);
```

შედეგი:

```
15
```

რა ხდება?

```
a = 5b = 10
```

და ითვლის:

```
5 + 10
```

return

ზოგჯერ ფუნქციას არა დაბეჭდვა, არამედ მნიშვნელობის დაბრუნება უნდა.

მაგალითი

```
function add(a, b){  
  return a + b;  
}  
let result = add(5, 10);  
console.log(result);
```

შედეგი:

```
15
```

განსხვავება

ეს:

```
console.log(a + b);
```

მხოლოდ ბეჭდავს.

ხოლო:

```
return a + b;
```

აბრუნებს მნიშვნელობას.

ფუნქცია მასივთან

```
function printArray(arr){
  for(let i = 0; i < arr.length; i++){
    console.log(arr[i]);
  }
  let nums = [10,20,30,40];
  printArray(nums);
}
```

რას აკეთებს?

მასივს გადასცემ:

```
nums
```

ფუნქცია კი ყველა ელემენტს ბეჭდავს.

prompt()-იანი ფუნქცია

```
function greetUser(){
  let name = prompt("შეიყვანე სახელი");
  console.log("გამარჯობა " + name);
}
greetUser();
```

ფუნქციის გამოყენება ჯამის დასათვლელად

```
function sumArray(arr){
  let sum = 0;
  for(let i = 0; i < arr.length; i++){
    sum += arr[i];
  }
}
```

```
    return sum;
  }
  let numbers = [10,20,30,40];
  console.log(sumArray(numbers));
```

1. `sort()` — მასივის დალაგება

`sort()` გამოიყენება მასივის ელემენტების დასალაგებლად.

ტექსტების დალაგება

```
let names = ["Luka", "Ana", "Giorgi", "Nino"];
names.sort();
console.log(names);
```

ახსნა

აქ გვაქვს სახელების მასივი:

```
["Luka", "Ana", "Giorgi", "Nino"]
```

როდესაც ვწერთ:

```
names.sort();
```

JavaScript ალაგებს ტექსტებს ანბანის მიხედვით.

შედეგი იქნება:

```
["Ana", "Giorgi", "Luka", "Nino"]
```

ანუ:

- Ana გადავიდა პირველ ადგილზე
- შემდეგ მოდის Giorgi
- შემდეგ Luka
- ბოლოს Nino

 რიცხვებზე `sort()` -ის პრობლემა

```
let nums = [100, 5, 20, 1];  
nums.sort();  
console.log(nums);
```

შედეგი იქნება:

```
[1, 100, 20, 5]
```

ეს იმიტომ ხდება, რომ ჩვეულებრივი `sort()` რიცხვებს ისე აღიქვამს, როგორც ტექსტს.

ანუ JavaScript ადარებს ასე:

```
"100", "5", "20", "1"
```

ამიტომ რიცხვებისთვის უნდა დავუწეროთ სპეციალური შედარების ფუნქცია.

სწორი დალაგება რიცხვებისთვის

```
let nums = [100, 5, 20, 1];  
nums.sort(function(a, b){  
    return a - b;  
});  
console.log(nums);
```

შედეგი:

```
[1, 5, 20, 100]
```

ახსნა

აქ:

```
function(a, b){    return a - b;}
```

ეუბნება JavaScript-ს:

დაალაგე რიცხვები ზრდადობით, ანუ პატარადან დიდისკენ.

თუ გვინდა კლებადობით:

```
let nums = [100, 5, 20, 1];
nums.sort(function(a, b){
  return b - a;
});
console.log(nums);
```

შედეგი:

```
[100, 20, 5, 1]
```

2. `find()` — ერთი ელემენტის მოძებნა

`find()` პოულობს პირველ ელემენტს, რომელიც აკმაყოფილებს პირობას.

```
let nums = [5, 12, 30, 7];
let result = nums.find(function(num){
  return num > 10;
});
console.log(result);
```

შედეგი:

```
12
```

ახსნა

მასივი არის:

```
[5, 12, 30, 7]
```

პირობა არის:

```
num > 10
```

JavaScript ამოწმებს ასე:

- `5 > 10` — `false`
- `12 > 10` — `true`

როგორც კი იპოვა პირველი სწორი ელემენტი, გაჩერდა და დააბრუნა `12`.

⚠ `find()` არ აბრუნებს ყველა შესაბამის ელემენტს. აბრუნებს მხოლოდ პირველს.

`find()` ტექსტებზე

```
let names = ["Nino", "Ana", "Giorgi", "Luka"];
let result = names.find(function(name){
  return name == "Giorgi";
});
console.log(result);
```

შედეგი:

Giorgi

თუ ელემენტი ვერ იპოვა:

```
let names = ["Nino", "Ana", "Luka"];
let result = names.find(function(name){
  return name == "Giorgi";});
console.log(result);
```

შედეგი:

undefined

`undefined` ნიშნავს: ვერ იპოვა.

3. `filter()` — რამდენიმე ელემენტის გაფილტვრა

`filter()` პოულობს ყველა ელემენტს, რომელიც პირობას აკმაყოფილებს და ქმნის ახალ მასივს.

```
let nums = [10, 25, 40, 7, 90];
let filtered = nums.filter(function(num){
  return num > 20;
});
console.log(filtered);
```

შედეგი:

```
[25, 40, 90]
```

ახსნა

პირობაა:

```
num > 20
```

JavaScript ამოწმებს:

- `10 > 20` — `false`, არ შედის ახალ მასივში
- `25 > 20` — `true`, შედის
- `40 > 20` — `true`, შედის
- `7 > 20` — `false`
- `90 > 20` — `true`, შედის

ამიტომ მივიღეთ:

```
[25, 40, 90]
```

`filter()` ლუწ რიცხვებზე

```
let nums = [1, 2, 3, 4, 5, 6];
let evenNums = nums.filter(function(num){
  return num % 2 == 0;
});
console.log(evenNums);
```

შედეგი:

```
[2, 4, 6]
```

აქ პირობაა:

```
num % 2 == 0
```

ეს ნიშნავს: რიცხვი 2-ზე იყოფა უნაშთოდ, ანუ ლუწია.

4. `map()` — მონაცემების ტრანსფორმაცია

`map()` იღებს მასივის თითოეულ ელემენტს, გარდაქმნის და აბრუნებს ახალ მასივს.

```
let nums = [1, 2, 3, 4];  
let doubled = nums.map(function(num){  
  return num * 2;  
});  
console.log(doubled);
```

შედეგი:

```
[2, 4, 6, 8]
```

ახსნა

მასივი იყო:

```
[1, 2, 3, 4]
```

პირობაა:

```
return num * 2;
```

ანუ:

- 1 → 2
- 2 → 4
- 3 → 6
- 4 → 8

ამიტომ მივიღეთ ახალი მასივი:

```
[2, 4, 6, 8]
```

⚠️ `map()` არ ფილტრავს. ის ყველა ელემენტს გარდაქმნის.

`map()` ტექსტებზე

```
let names = ["nino", "giorgi", "ana"];
let upperNames = names.map(function(name){
  return name.toUpperCase();
});
console.log(upperNames);
```

შედეგი:

```
["NINO", "GIORGI", "ANA"]
```

აქ თითოეული სახელი გარდაიქმნა დიდ ასოებად.

მთავარი განსხვავება

მეთოდი	რას აკეთებს	აბრუნებს
<code>sort()</code>	ალაგებს მასივს	დალაგებულ მასივს
<code>find()</code>	ეძებს პირველ შესაბამის ელემენტს	ერთ ელემენტს
<code>filter()</code>	ტოვებს ყველა შესაბამის ელემენტს	ახალ მასივს
<code>map()</code>	გარდაქმნის ყველა ელემენტს	ახალ მასივს

External JavaScript მაგალითი

`index.html`

```
<!DOCTYPE html><html><head>
  <title>Lesson 6</title>
</head><body>
  <h1>Array Methods</h1>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
let nums = [];
for(let i = 0; i < 5; i++){
  let n = Number(prompt("შეყვანე რიცხვი"));
  nums.push(n);
}
console.log("შეყვანილი მასივი:", nums);
let sortedNums = nums.sort(function(a, b){
  return a - b;});
console.log("დალაგებული:", sortedNums);
let firstBig = nums.find(function(num){
  return num > 50;});
console.log("პირველი 50-ზე მეტი:", firstBig);
let evenNums = nums.filter(function(num){
  return num % 2 == 0;});
console.log("ლუწები:", evenNums);
let doubledNums = nums.map(function(num){
  return num * 2;});
console.log("გაორმაგებული:", doubledNums);
```

სავარჯიშოები

მარტივი

1. შექმენი სახელების მასივი და დაალაგე ანბანის მიხედვით `sort()` -ით.
2. შექმენი რიცხვების მასივი და იპოვე პირველი რიცხვი, რომელიც 10-ზე მეტია `find()` -ით.
3. შექმენი რიცხვების მასივი და დატოვე მხოლოდ ლუწები `filter()` -ით.
4. შექმენი რიცხვების მასივი და ყველა რიცხვი გაამრავლე 3-ზე `map()` -ით.

საშუალო

1. prompt-ით შეაყვანინე მომხმარებელს 6 რიცხვი და დაალაგე ზრდადობით.
2. prompt-ით შეაყვანინე 5 სახელი და იპოვე პირველი სახელი, რომელიც "A" -ზე იწყება.
3. შექმენი ქულების მასივი და დატოვე მხოლოდ 51-ზე მეტი ქულები.
4. შექმენი ფასების მასივი და ყველა ფასს დაუმატე 5 ლარი.

რთული

1. მომხმარებელს შეაყვანინე 7 რიცხვი. შემდეგ:
 - დაალაგე ზრდადობით
 - იპოვე პირველი 100-ზე მეტი
 - დატოვე მხოლოდ ლუწები
 - შექმენი ახალი მასივი, სადაც ყველა რიცხვი გაორმაგებულია
2. მომხმარებელს შეაყვანინე 6 სახელი. შემდეგ:
 - დაალაგე ანბანის მიხედვით
 - იპოვე პირველი სახელი, რომლის სიგრძე 5-ზე მეტია
 - დატოვე მხოლოდ ის სახელები, რომლებიც "N" -ზე იწყება
 - შექმენი ახალი მასივი, სადაც ყველა სახელი დიდი ასოებით იქნება.